ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT III

Knowledge Representation and Reasoning

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

Knowledge representation issues, predicate logic, logic programming, semantic nets, frames and inheritance, constraint propagation, rule-based knowledge and deduction systems. Reasoning under uncertainty: probability review, Bayes' inference, Dempster-Shafer theory.

1. Knowledge Representation: Issues

An intelligent system must **know** things (facts, rules, relations) and be able to **use** that knowledge. **Knowledge representation (KR)** is the study of how to encode knowledge in a form a computer can store and reason with.

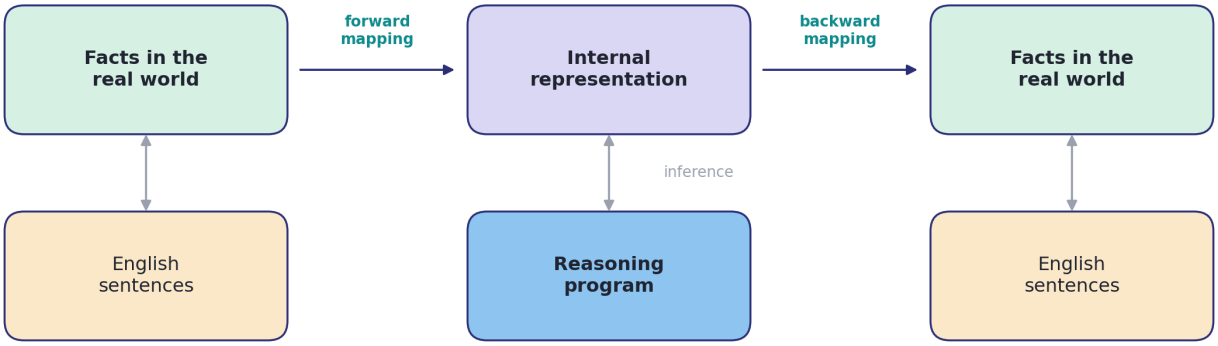


Fig 3.1 Mapping between facts and their representation: reasoning happens on the representation

What makes a good representation? Four properties

Property	Meaning
Representational adequacy	Can it represent *all* the kinds of knowledge needed in the domain?
Inferential adequacy	Can it derive *new* knowledge from the existing structures?
Inferential efficiency	Can the inference be guided toward the most promising directions?
Acquisitional efficiency	Can new knowledge be added easily (automatically if possible)?

Types of knowledge

- **Declarative:** facts about the world ("Paris is the capital of France").
- **Procedural:** how to do things (rules, plans, algorithms).
- **Meta-knowledge:** knowledge about knowledge (which rule to trust, how reliable a fact is).
- **Heuristic:** rules of thumb gained from experience.

Approaches to representing knowledge

Approach	Idea	Example
Simple relational	Facts stored as tables of attributes	A database of employees
Inheritable	Objects in a class hierarchy inherit properties	Semantic nets, frames
Inferential	Facts plus rules for deduction	Predicate logic, rules
Procedural	Knowledge embedded in executable code	Programs, production rules

Key issues in knowledge representation

- **Important attributes:** basic attributes such as *instance* and *isa* support inheritance and appear in almost every domain.
- **Relationships among attributes:** inverses (e.g., *parent / child*), existence in an isa hierarchy, single-valued attributes.
- **Granularity:** at what level of detail should knowledge be represented ("John spotted Sue" or a full description of vision)?
- **Representing sets of objects:** as one set (class) with properties, or individually?
- **Finding the right structure:** selecting the relevant piece of knowledge from a huge store when needed.

2. Predicate Logic

Predicate logic (first-order predicate calculus) is a precise, expressive language for knowledge. Where propositional logic only says "P is true", predicate logic describes **objects, their properties and relations**, and can state facts about *all* or *some* objects.

Quick recap: propositional logic

Propositional logic deals with whole statements (propositions) that are either true or false, combined with connectives. It is the foundation on which predicate logic is built.

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ↔ Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Inference rule	Pattern	Example
Modus Ponens	P, P → Q ⊢ Q	It rains; if it rains the road is wet ⊢ road is wet
Modus Tollens	¬Q, P → Q ⊢ ¬P	Road is dry; if rain then wet ⊢ it is not raining
And-elimination	P ∧ Q ⊢ P	Ram is tall and strong ⊢ Ram is tall
Hypothetical syllogism	P → Q, Q → R ⊢ P → R	Study → pass; pass → degree ⊢ study → degree

Limitation: propositional logic cannot say "*all* students are smart" without listing every student. Predicate logic fixes this with variables and quantifiers.

Building blocks

Element	Notation	Example
Constants (objects)	Marcus, Caesar, 5	Marcus
Variables	x, y, z	x
Predicates (relations)	man(x), loves(x, y)	man(Marcus)
Functions	father(x)	father(Ravi)
Connectives	∧ and, ∨ or, ¬ not, → implies, ↔ iff	man(x) ∧ mortal(x)
Quantifiers	∀ for all, ∃ there exists	∀x man(x) → mortal(x)

REMEMBER
∀ usually goes with **→**: "All students are smart" = ∀x (student(x) → smart(x)).
∃ usually goes with **∧**: "Some student is smart" = ∃x (student(x) ∧ smart(x)).
Negation: ¬∀x P(x) ≡ ∃x ¬P(x) and ¬∃x P(x) ≡ ∀x ¬P(x).

English to predicate logic

English sentence	Predicate logic
Marcus was a man.	man(Marcus)
All Pompeians were Romans.	∀x: Pompeian(x) → Roman(x)
Every Roman was loyal to Caesar or hated him.	∀x: Roman(x) → loyalto(x, Caesar) ∨ hate(x, Caesar)
Everybody is loyal to someone.	∀x ∃y: loyalto(x, y)
Nobody likes taxes.	¬∃x: likes(x, taxes)

EXAMPLE
Inference by Modus Ponens. Given: (1) Pompeian(Marcus), (2) ∀x Pompeian(x) → Roman(x). Substitute x = Marcus in (2): Pompeian(Marcus) → Roman(Marcus). With (1), conclude **Roman(Marcus)**.

Logic programming (Prolog)

Logic programming uses a restricted form of logic (Horn clauses) as a programming language. The most famous language is **Prolog**. A program is a set of **facts** and **rules**; running it means asking a **query**, which the system answers by **backward chaining with backtracking** (search + unification).

```
parent(tom, bob).                 % fact: Tom is a parent of Bob
parent(bob, ann).                 % fact
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).   % rule: X is grandparent of Z
                                                   %   IF X is parent of Y AND Y is parent of Z
?- grandparent(tom, W).           % query
W = ann.                          % answer
```

Prolog term	Meaning
Fact	A clause that is unconditionally true: *head.*
Rule	*head :- body.* Head is true if all goals in the body are true (**:-** means IF, comma means AND)
Query	*?- goal.* Asks whether the goal follows from the program, and finds variable bindings
Variable	Starts with a capital letter (X, Y); constants and predicates start with lower case

3. Semantic Nets

A **semantic network** is a graph of knowledge: **nodes** represent objects, concepts or situations, and **labelled arcs** represent the relationships between them. The two most important links are **is-a** (a class is a subclass of another) and **instance-of** (an object belongs to a class).

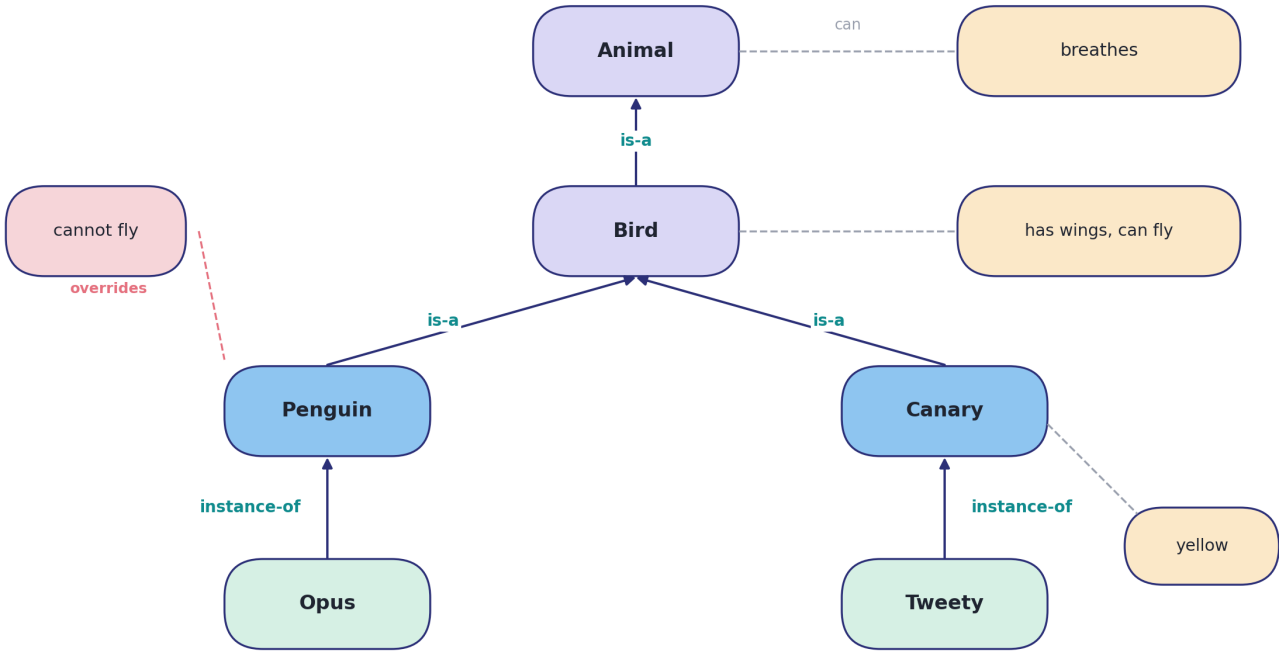


Fig 3.2 Semantic network: nodes are concepts/objects, labelled arcs are relations; properties are inherited down the is-a links

Inheritance

Properties stored at a general node are **inherited** by everything below it. Because Tweety is an instance of Canary, which is-a Bird, we can deduce *Tweety has wings and can fly* without storing it separately. A more specific node can **override** an inherited value: Penguin is-a Bird but *cannot fly*, so Opus does not fly.

Advantages	Limitations
Easy to visualise; natural for hierarchies	No standard meaning for links, so interpretation can be ambiguous
Efficient inheritance reduces storage	Hard to represent negation, disjunction and quantifiers properly
Quick access to related concepts by following links	Exceptions and defaults can cause inconsistent conclusions

4. Frames and Inheritance

A **frame** (Marvin Minsky, 1975) is a structured record describing a stereotyped object or situation. It is like a class: it has a **name** and a collection of **slots** (attributes), and each slot holds a value and possibly extra information called **facets**.

Facet	Purpose
Value	The actual filler of the slot
Default	Value assumed when no other value is known
Range / type	Restricts which values are legal
If-needed (procedure)	Code executed to *compute* the value when asked
If-added / if-removed (demon)	Code triggered automatically when the value changes

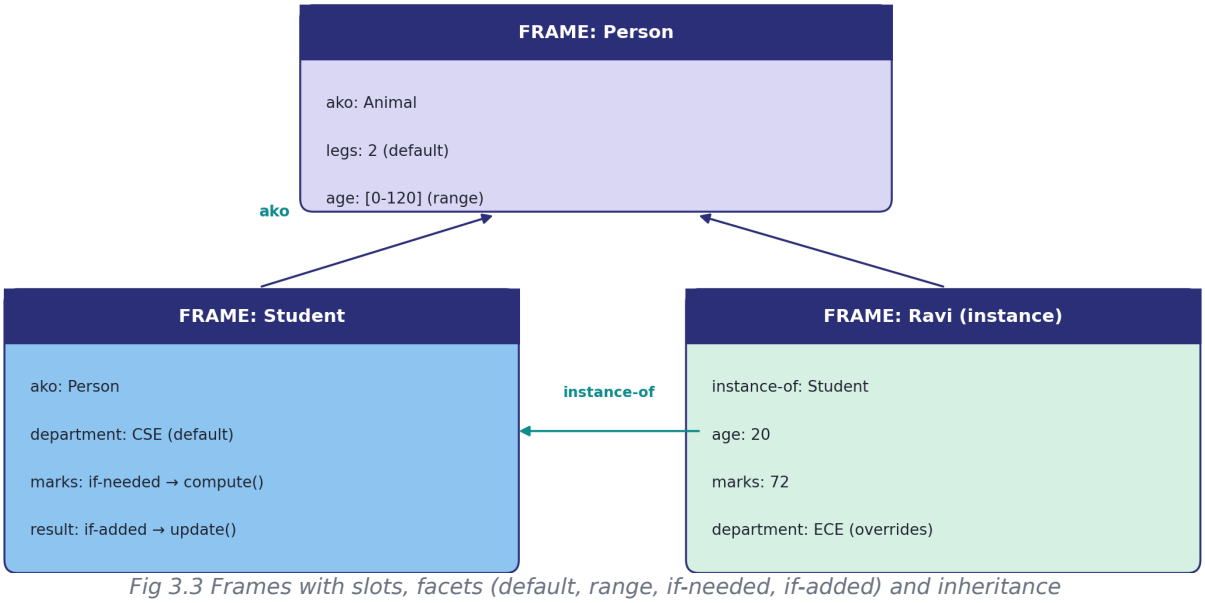


Fig 3.3 Frames with slots, facets (default, range, if-needed, if-added) and inheritance

- **Inheritance:** a child frame (linked by *ako*, 'a kind of', or *instance-of*) inherits slots and defaults from its parent frames. Local values override inherited defaults.
- **Example:** Ravi is an instance of Student, so he inherits *legs = 2* from Person. His department is stored locally as ECE, which overrides the default CSE.
- **Frames vs semantic nets:** frames group all information about one object in one place and add procedures and defaults; semantic nets spread information over graph links.

Choosing a representation: quick comparison

Scheme	Best for	Weakness
Predicate logic	Precise facts and sound deduction	Can be slow; hard to handle exceptions/uncertainty
Semantic nets	Class hierarchies, relations between concepts	No formal semantics; poor at quantifiers
Frames	Stereotyped objects and situations with defaults	Rigid structure; defaults can conflict
Production rules	Expert knowledge, modular and easy to edit	Large rule bases are hard to maintain
Probabilistic models	Reasoning with uncertain evidence	Needs many probabilities

5. Constraint Propagation

Many problems (timetabling, map colouring, Sudoku, scheduling) are **constraint satisfaction problems (CSPs)**. A CSP has three parts:

- **Variables** $X_1 \ldots X_n$
- **Domains** D_i, the set of possible values of each variable
- **Constraints**, rules saying which combinations of values are allowed

Constraint propagation uses the constraints to **reduce the domains** of variables: whenever a value is assigned or removed, its consequences are spread to neighbouring variables, pruning values that can no longer be part of a solution. This often finds the answer with little or no search.

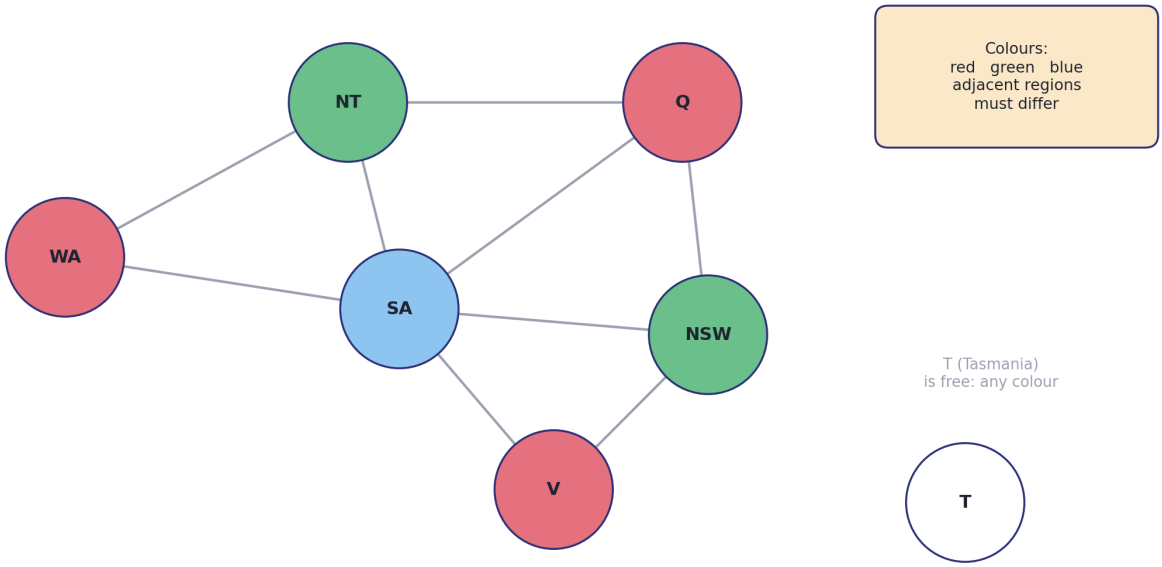


Fig 3.4 Constraint graph for map colouring (nodes = variables, edges = 'must differ' constraints), shown with a valid solution

Step	Action	Domains after propagation
0	Start	All variables {R, G, B}
1	Assign WA = R	NT {G, B}; SA {G, B}
2	Assign NT = G	SA {B}; Q {R, B}
3	SA is forced to B	Q {R}; NSW {R, G}; V {R, G}
4	Q is forced to R	NSW {G}
5	NSW is forced to G	V {R} (T may be any colour)

- **Forward checking:** after assigning a variable, delete inconsistent values from its neighbours' domains. If any domain becomes empty, **backtrack** immediately.
- **Arc consistency:** an arc X → Y is consistent if for *every* value of X there is some allowed value of Y. Inconsistent values are deleted repeatedly (the AC-3 algorithm) until all arcs are consistent.
- **Heuristics:** choose the variable with the *fewest remaining values* (MRV) first, and try the least constraining value first.

EXAM TIP
Constraint propagation = **inference** that narrows domains. Backtracking = **search**. Real CSP solvers interleave both.

6. Representing Knowledge Using Rules

The most popular way to hold expert knowledge is the **production rule**, an IF-THEN statement: **IF** conditions (antecedent) **THEN** action or conclusion (consequent). A system built from such rules is a **production system**.

EXAMPLE
R1: IF has_feathers AND lays_eggs THEN bird
R2: IF bird AND cannot_fly AND swims THEN penguin
R3: IF fever AND cough THEN suspect_flu

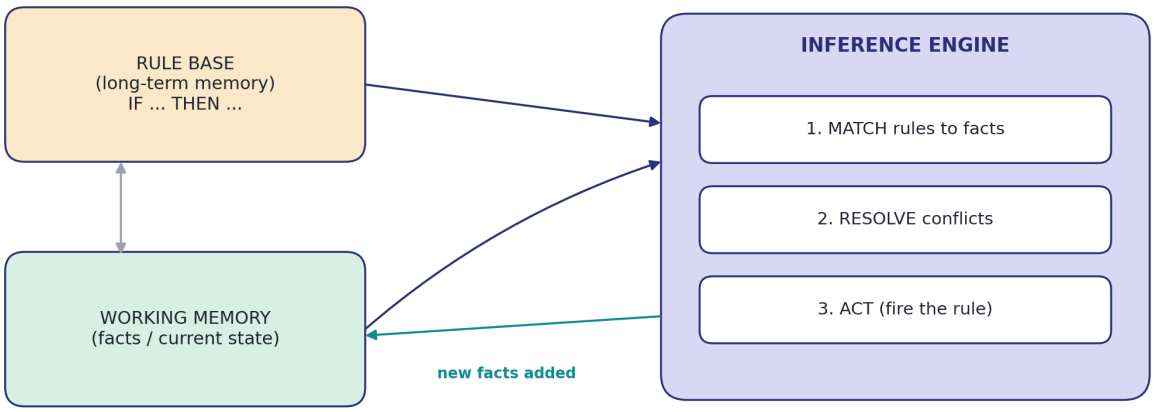


Fig 3.5 Architecture of a production (rule-based) system: match, resolve, act

Components of a production system

- **Rule base** (production memory): the set of IF-THEN rules, i.e., the permanent knowledge.
- **Working memory:** the facts that are currently known; it changes as rules fire.
- **Inference engine** (control strategy): repeatedly **matches** rules against the working memory, picks one to fire (**conflict resolution**), and **acts** on it.

Conflict resolution strategies

Strategy	Idea
Specificity	Prefer the rule with more conditions (the more specific rule)
Recency	Prefer rules that use the most recently added facts
Rule order	Choose the first matching rule in the list (priority)
Refractoriness	Never fire the same rule twice on the same facts

7. Rule-Based Deduction Systems

These systems derive conclusions from the rules using one of two directions of reasoning.

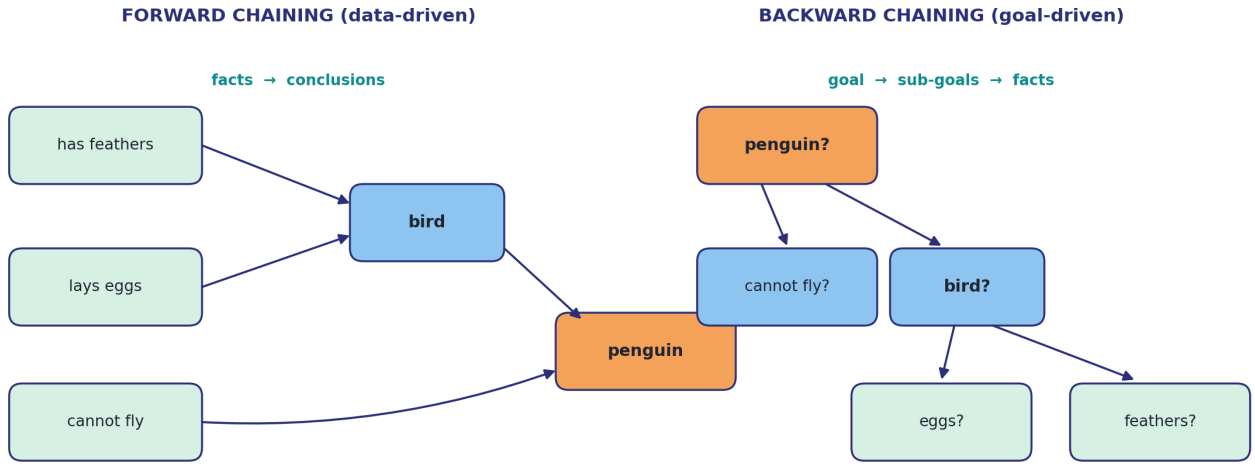


Fig 3.6 Forward vs backward chaining on the same rules

	Forward chaining	Backward chaining
Direction	From known facts to conclusions	From a goal back to supporting facts
Also called	Data-driven, bottom-up	Goal-driven, top-down
Starts with	Facts in working memory	A hypothesis (goal) to prove
Good when	Many facts, no single goal: monitoring, design, planning	A specific goal: diagnosis, 'is X true?'
Example systems	XCON (R1), OPS5	MYCIN, Prolog

EXAMPLE
Forward: facts {has_feathers, lays_eggs, cannot_fly, swims}. R1 fires → add *bird*. R2 fires → add *penguin*.
Backward: goal *penguin?* → needs *bird*, *cannot_fly*, *swims*. To prove *bird* → R1 needs has_feathers and lays_eggs, both in facts. Goal proved.

8. Reasoning under Uncertainty

Real-world knowledge is rarely certain. A doctor sees symptoms without being sure of the disease; a sensor gives noisy readings. Pure logic (true/false) cannot cope, so AI uses methods that handle **degrees of belief**.

Sources of uncertainty	Example
Incomplete information	Not all test results are available
Noisy / unreliable data	A faulty sensor reading
Imprecise knowledge	"Fever usually means infection"
Randomness in the world	Will it rain tomorrow?

8.1 Review of probability

- **Probability P(A)** $\in [0, 1]$. P(true) = 1, P(false) = 0, P(¬A) = 1 − P(A). All outcomes sum to 1.
- **Joint probability** P(A ∧ B): both occur. **Union:** P(A ∨ B) = P(A) + P(B) − P(A ∧ B).
- **Conditional probability:** P(A | B) = P(A ∧ B) / P(B), the chance of A once B is known.
- **Independence:** A and B are independent if P(A ∧ B) = P(A)·P(B), equivalently P(A | B) = P(A).
- **Product rule:** P(A ∧ B) = P(A | B)·P(B).

8.2 Bayes' theorem and probabilistic inference

FORMULA

P(H | E) = P(E | H) · P(H) / P(E)

P(H) = **prior** belief in hypothesis H | P(E | H) = **likelihood** of the evidence if H is true | P(H | E) = **posterior** belief after seeing E.

P(E) = P(E | H)P(H) + P(E | ¬H)P(¬H)

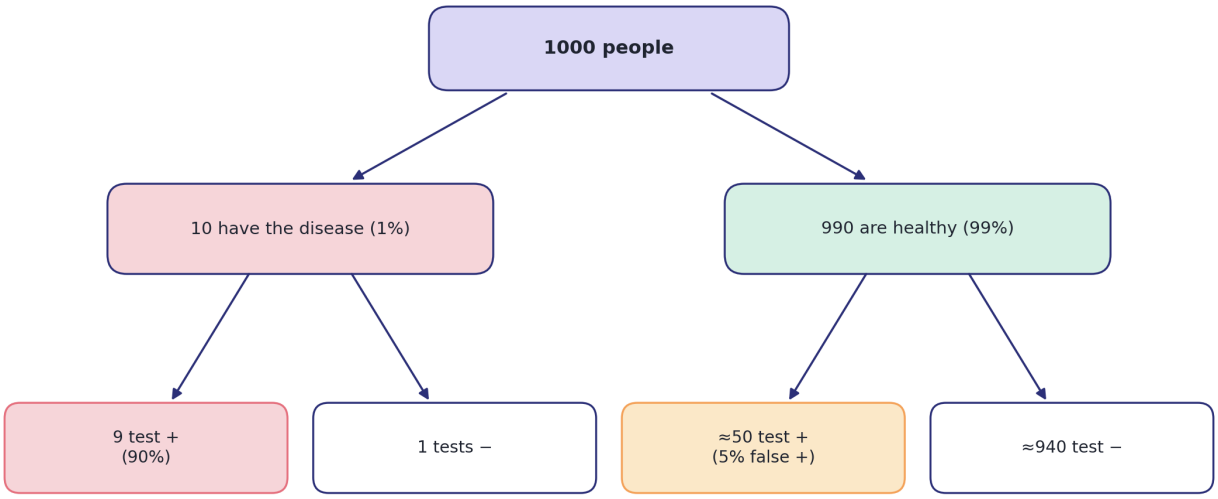
Worked example. 1% of people have a disease. A test is positive for 90% of sick people (sensitivity) and for 5% of healthy people (false positives). A person tests positive. What is the chance they are sick?

SOLUTION

P(D) = 0.01, P(+ | D) = 0.90, P(+ | ¬D) = 0.05.

P(+) = 0.90 × 0.01 + 0.05 × 0.99 = 0.009 + 0.0495 = 0.0585.

P(D | +) = 0.009 / 0.0585 ≈ 0.154 (about 15%). Despite a 90% accurate test, most positives are false alarms because the disease is rare (low prior).



Positives = 9 + 50 = 59. Truly diseased among them = 9/59 ≈ 15% (not 90%!)

Fig 3.7 Natural-frequency view of Bayes' theorem (disease 1%, sensitivity 90%, false-positive rate 5%)

EXAMPLE

Factory problem. Machine M1 makes 60% of items with 3% defective; M2 makes 40% with 5% defective. An item is found defective. Which machine made it?
P(D) = 0.6×0.03 + 0.4×0.05 = 0.018 + 0.020 = 0.038. P(M1 | D) = 0.018/0.038 ≈ **0.474**; P(M2 | D) = 0.020/0.038 ≈ **0.526**.

Naive Bayes: with several pieces of evidence $E_1 \ldots E_n$ assumed independent given H, $P(H \mid E_1 \ldots E_n) \propto P(H) \cdot \Pi P(E_i \mid H)$. It is simple and works surprisingly well (spam filters, diagnosis).

Bayesian (belief) networks

A **Bayesian network** is a directed acyclic graph in which each node is a random variable and each arrow means 'directly influences'. Each node stores a **conditional probability table** given its parents. The network gives the full joint distribution compactly: $\mathbf{P(x_1 \ldots x_n) = \Pi\ P(x_i \mid parents(x_i))}$.

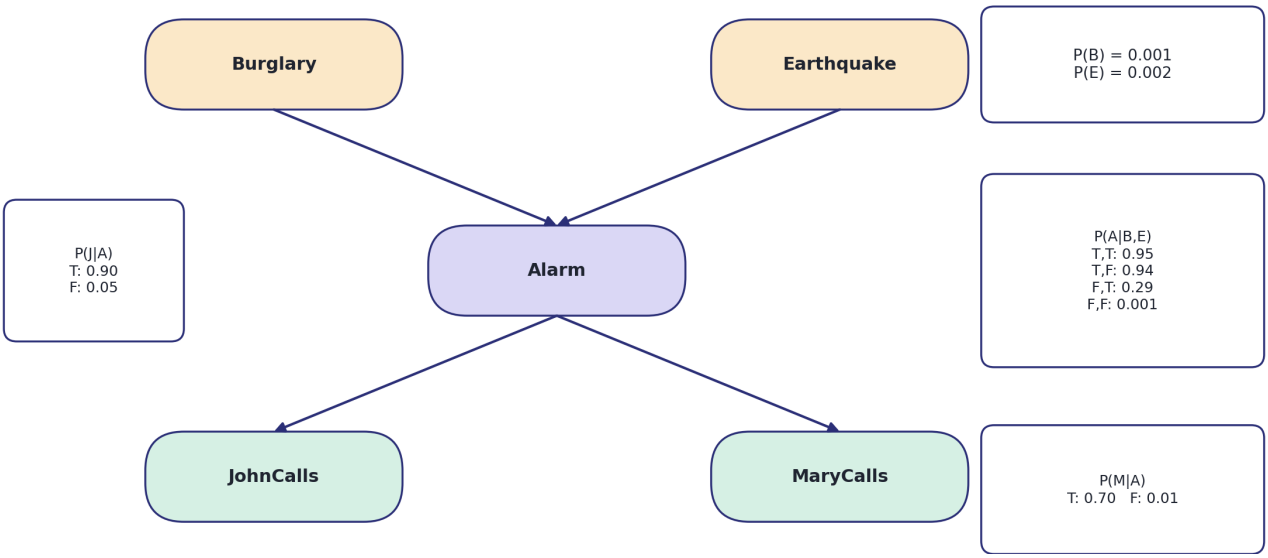


Fig 3.8 A Bayesian network: arrows show direct influence; each node stores a conditional probability table (CPT)

EXAMPLE

P(John calls, Mary calls, alarm, no burglary, no earthquake) = P(J|A)·P(M|A)·P(A|¬B,¬E)·P(¬B)·P(¬E) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 ≈ **0.00063**.

8.3 Dempster-Shafer theory

Bayes' theory forces you to assign probabilities to *every* hypothesis, even when you know nothing. **Dempster-Shafer (D-S) theory** allows you to say 'I don't know' by assigning belief to **sets** of hypotheses and keeping the leftover as **ignorance**.

Term	Meaning
Frame of discernment Θ	The set of all mutually exclusive possible answers, e.g., Θ = {A, B}
Mass function m	Assigns a mass to each subset of Θ; masses sum to 1 and m(∅) = 0. m(Θ) represents ignorance
Belief Bel(X)	Total mass of all subsets of X: the evidence that *supports* X (lower bound)
Plausibility Pl(X)	1 − Bel(¬X): the evidence that does *not rule out* X (upper bound)
Belief interval	[Bel(X), Pl(X)]: the width Pl − Bel measures the uncertainty about X

Dempster's rule of combination merges two independent bodies of evidence: $m(Z) = \Sigma_{X \cap Y = Z} m_1(X) \cdot m_2(Y) / (1 - K)$, where K = the total mass that falls on empty intersections (the **conflict**).

EXAMPLE

Θ = {A, B}. Evidence 1: $m_1(\{A\}) = 0.6$, $m_1(\Theta) = 0.4$. Evidence 2: $m_2(\{A\}) = 0.5$, $m_2(\{B\}) = 0.3$, $m_2(\Theta) = 0.2$.

Conflict: {A}∩{B} = ∅, so K = 0.6 × 0.3 = 0.18, and 1 − K = 0.82.

m({A}) = (0.6×0.5 + 0.6×0.2 + 0.4×0.5) / 0.82 = 0.62/0.82 ≈ **0.756**; m({B}) = (0.4×0.3)/0.82 ≈ **0.146**; m(Θ) = (0.4×0.2)/0.82 ≈ **0.098**.

Bel(A) = 0.756; Pl(A) = 1 − Bel(B) = 1 − 0.146 = 0.854. Belief interval for A = **[0.756, 0.854]**.

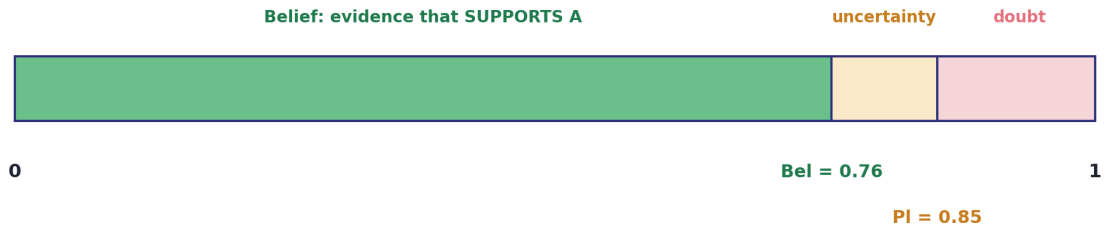


Fig 3.9 Belief interval [Bel(A), Pl(A)] for the Dempster-Shafer example

	Bayesian probability	Dempster-Shafer
Ignorance	Must be spread over hypotheses (e.g., 50/50)	Kept explicitly as mass on Θ
Output	A single probability P(H)	An interval [Bel, Pl]
Evidence about sets	Only about single outcomes	Can be assigned to sets of outcomes
Needs	Prior probabilities and likelihoods	Mass functions only

Quick Revision: Unit III at a Glance

Topic	One-line recall
KR properties	Representational and inferential adequacy, inferential efficiency, acquisitional efficiency
Predicate logic	Objects, predicates, ∀ with →, ∃ with ∧; reasoning by Modus Ponens
Prolog	Facts + rules (head :- body) + queries, solved by backward chaining
Semantic net / frame	Graph of concepts with is-a links / structured slots with defaults and demons; both inherit
Constraint propagation	Spread constraints to shrink domains; forward checking, arc consistency
Production system	Rule base + working memory + inference engine (match, resolve, act)
Forward / backward	Data-driven from facts / goal-driven from hypothesis
Bayes	P(H|E) = P(E|H)P(H)/P(E)
Dempster-Shafer	Masses on sets; belief interval [Bel, Pl]; combine with Dempster's rule

Frequently asked exam questions

- Discuss the issues in knowledge representation and the properties of a good KR scheme.
- Represent given English statements in predicate logic. Write a short Prolog program.
- Explain semantic nets and frames with examples. How does inheritance work?
- What is constraint propagation? Explain with a map-colouring example.
- Explain a rule-based deduction system. Compare forward and backward chaining.
- State Bayes' theorem and solve a numerical problem. Explain Dempster-Shafer theory with an example.